

Introducing students to the language first, and then showing them how to design digital systems with the languages, tends to confuse students. Beginning student need not understand the details of VHDL -instead they should be able to modify examples to build the desired basic circuits. Students learn the importance of HDL-based digital design, without having to learn the complexities of HDLs. Those complexities can be reserved for a second, more advanced course.

Start from basic gates and work their way up to a simple microprocessor.

Examples have been simulated by Simulator and synthesized with Synthesis tool. Several design examples been tested on Xilinx ISE Software and Xilinx Spartan 3E FPGA Board.

Basic Logic Gates

Every VHDL design description consists of at least one entity/architecture pair, or one entity with multiple architectures. The entity section of the HDL design is used to declare the I/O ports of the circuit, while the description code resides within architecture portion. Standardized design libraries are typically used and are included prior to the entity declaration. This is accomplished by including the code "library ieee;" and "use ieee.std_logic_1164.all;".

1. Driver

-- two descriptions provided

library ieee;
use ieee.std_logic_1164.all;

entity Driver is
port(x: in std_logic;
 F: out std_logic
);
end Driver;

architecture behv1 of Driver is
begin

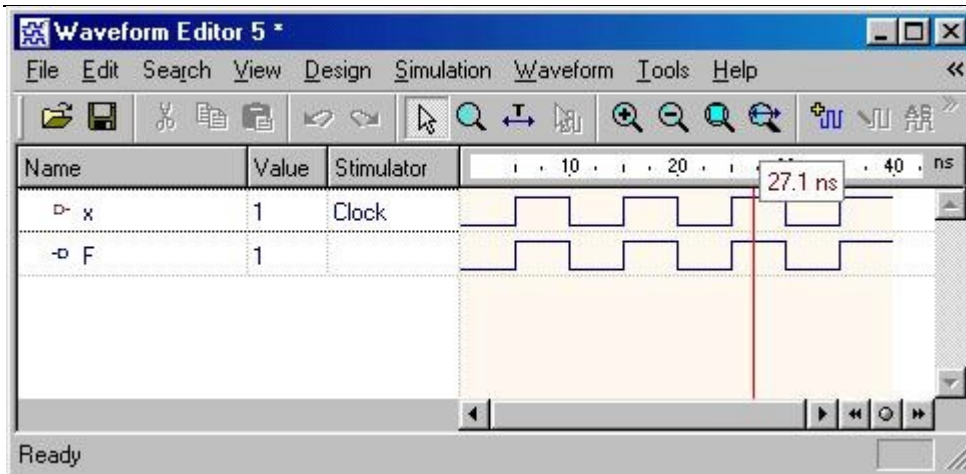
 process(x)
 begin
 -- compare to truth table
 if (x='1') then
 F <= '1';
 else
 F <= '0';
 end if;
 end process;

```
end behv1;
```

```
architecture behv2 of Driver is  
begin
```

```
    F <= x;
```

```
end behv2;
```



2. OR

```
-----  
-- OR gate (ESD book figure 2.3)  
-- two descriptions provided  
-----
```

```
library ieee;  
use ieee.std_logic_1164.all;  
-----
```

```
entity OR_ent is  
port(   x: in std_logic;  
        y: in std_logic;  
        F: out std_logic  
);  
end OR_ent;
```

```
-----  
architecture OR_arch of OR_ent is  
begin  
    process(x, y)  
    begin  
        -- compare to truth table  
        if ((x='0') and (y='0')) then  
            F <= '0';  
        else  
            F <= '1';  
        end if;  
    end process;  
end OR_arch;
```

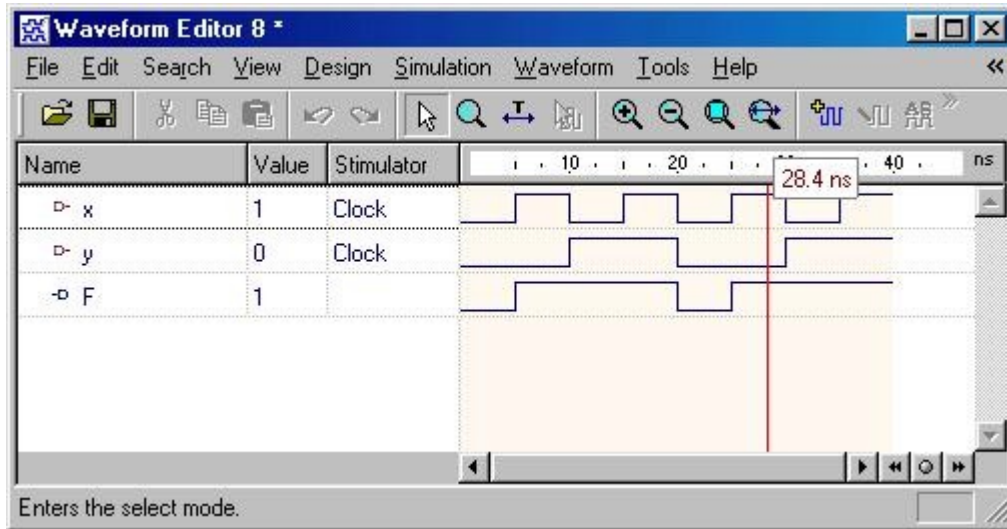
```

    end process;

end OR_arch;

architecture OR_beh of OR_ent is
begin
    F <= x or y;
end OR_beh;

```



3. AND

```

-- AND gate (ESD book figure 2.3)
-- two descriptions provided

```

```

library ieee;
use ieee.std_logic_1164.all;

```

```

entity AND_ent is
port(   x: in std_logic;
        y: in std_logic;
        F: out std_logic
);
end AND_ent;

```

```

architecture behav1 of AND_ent is
begin

    process(x, y)
    begin

```

```

-- compare to truth table
if ((x='1') and (y='1')) then
    F <= '1';
else
    F <= '0';
end if;
end process;

```

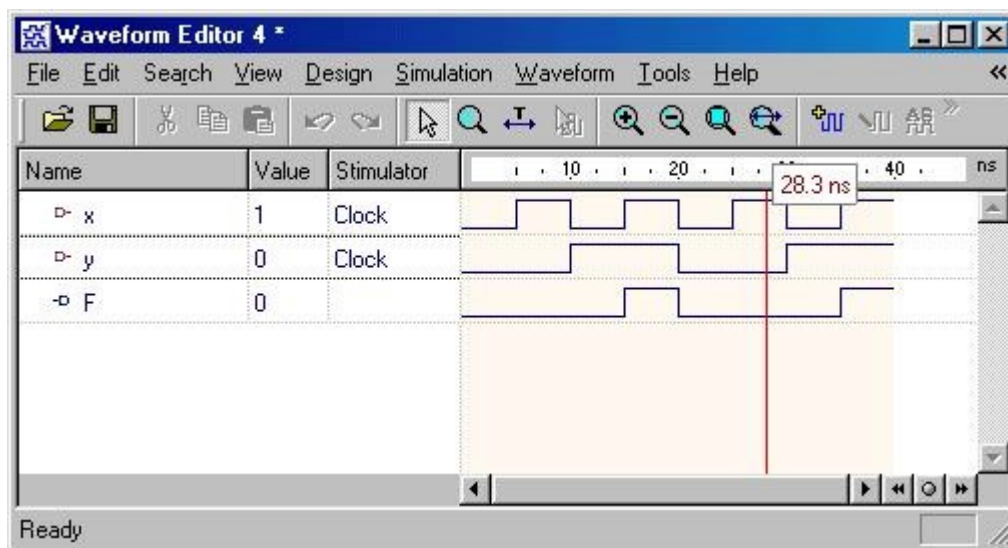
```
end behav1;
```

```
architecture behav2 of AND_ent is
begin

```

```
    F <= x and y;
```

```
end behav2;
```



4. XOR

```

-- XOR gate (ESD figure 2.3)
--
-- two descriptions provided
-----

```

```

library ieee;
use ieee.std_logic_1164.all;
-----

```

```

entity XOR_ent is
port(    x: in std_logic;

```

```

        y: in std_logic;
        F: out std_logic
    );
end XOR_ent;

```

architecture behv1 of XOR_ent is
begin

```

    process(x, y)
    begin
        -- compare to truth table
        if (x/=y) then
            F <= '1';
        else
            F <= '0';
        end if;
    end process;

```

end behv1;

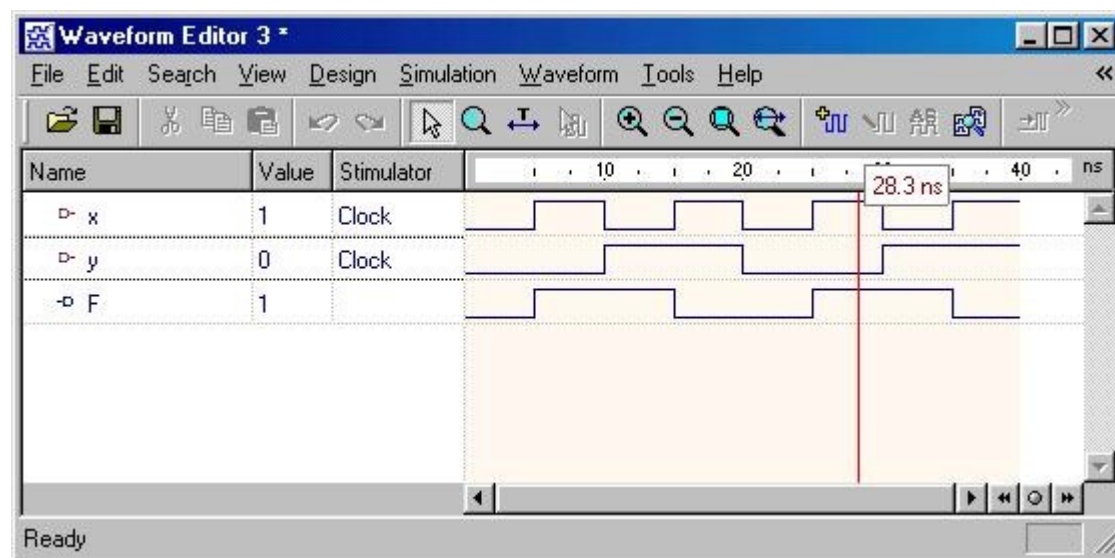
architecture behv2 of XOR_ent is
begin

```

    F <= x xor y;

```

end behv2;



Inverter

5. NOR

```
-----  
-- NOR gate (ESD figure 2.3)  
--  
-- two descriptions provided  
-----
```

```
library ieee;  
use ieee.std_logic_1164.all;  
  
-----
```

```
entity NOR_ent is  
port(   x: in std_logic;  
        y: in std_logic;  
        F: out std_logic  
);  
end NOR_ent;
```

```
-----  
architecture behv1 of NOR_ent is  
begin
```

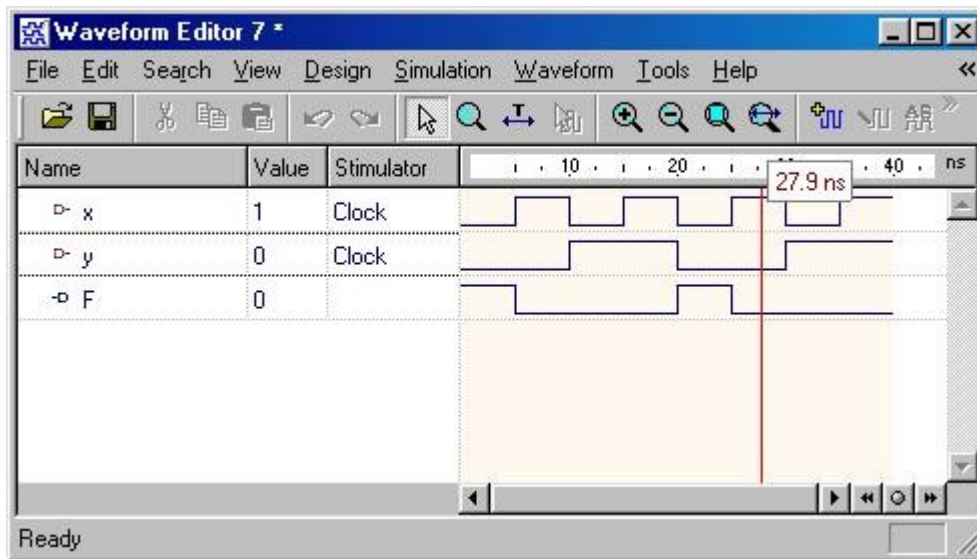
```
    process(x, y)  
    begin  
        -- compare to truth table  
        if (x='0' and y='0') then  
            F <= '1';  
        else  
            F <= '0';  
        end if;  
    end process;
```

```
end behv1;
```

```
architecture behv2 of NOR_ent is  
begin
```

```
    F <= x nor y;
```

```
end behv2;  
  
-----
```



6. NAND

 -- NAND gate (ESD figure 2.3)
 --
 -- two descriptions provided

library ieee;
 use ieee.std_logic_1164.all;

 entity NAND_ent is
 port(x: in std_logic;
 y: in std_logic;
 F: out std_logic
);
 end NAND_ent;

 architecture behv1 of NAND_ent is
 begin

process(x, y)
 begin
 -- compare to truth table
 if (x='1' and y='1') then
 F <= '0';
 else
 F <= '1';
 end if;
 end process;

```

        end if;
    end process;

```

```

end behv1;

```

```

-----

architecture behv2 of NAND_ent is
begin

```

```

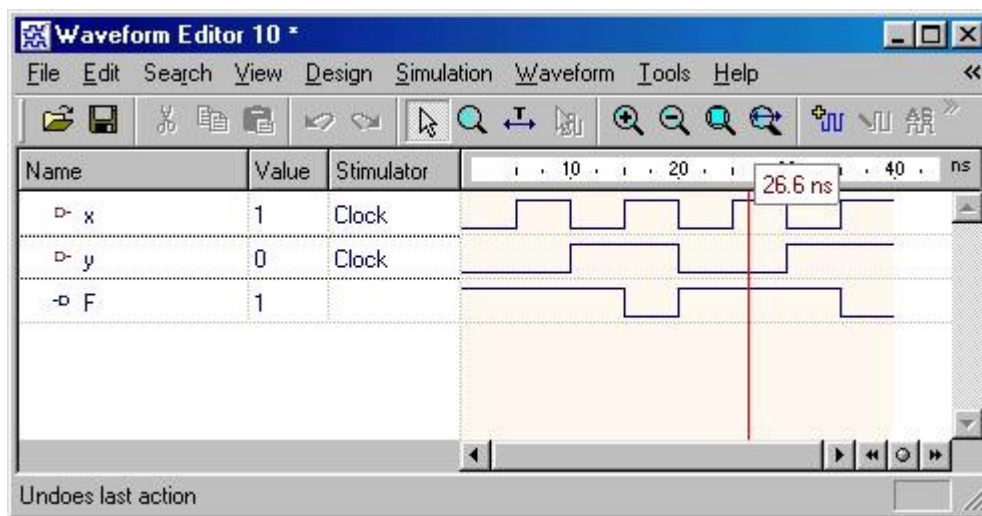
    F <= x nand y;

```

```

end behv2;

```



7. XNOR

```

-- XNOR gate (ESD figure 2.3)

```

```

--

```

```

-- two descriptions provided

```

```

-----

library ieee;
use ieee.std_logic_1164.all;

```

```

-----

entity XNOR_ent is
port(
    x: in std_logic;
    y: in std_logic;
    F: out std_logic
);
end XNOR_ent;

```

architecture behv1 of XNOR_ent is
begin

```
process(x, y)
begin
  -- compare to truth table
  if (x/=y) then
    F <= '0';
  else
    F <= '1';
  end if;
end process;
```

end behv1;

architecture behv2 of XNOR_ent is
begin

```
F <= x xnor y;
```

end behv2;

